Università degli Studi di Salerno

Corso di Ingegneria del Software

WeBooks
Problem Statement
Versione 1.1



Data: 14/10/2026

Coordinatore del progetto:

Nome	Matricola
Cuono Crimaldi	0512116728
Gaetano Avella	0512122083

Partecipanti:

Nome	Matricola
Cuono Crimaldi	0512116728
Gaetano Avella	0512122083

Scritto da:	Cuono Crimaldi, Gaetano Avella

Revision History

Data	Versione	Descrizione	Autore
12/10/2026	1.0	Prima creazione del Problem Statement	Cuono Crimaldi, Gaetano Avella
14/10/2026	1.1	Revisione requisiti, dettaglio ambiente target e aggiornamento scadente	Cuono Crimaldi, Gaetano Avella

Indice

1. Dominio del problema 4
2. Obiettivi 4
3. Scenari 5
   3.1 Scenario 1 5
   3.2 Scenario 2 5
   3.3 Scenario 3 6
4. Requisiti funzionali 6
   4.1. Funzionalità per l'attore Lettore (Cliente) 6
   4.2. Funzionalità per l'attore Amministratore (Libraio/Gestore) 7
5. Requisiti Non Funzionali 7
6. Ambiente Target 8
7. Consegne e scadenze 8

1. Dominio del problema

Attualmente, il mercato online dei libri è fortemente frammentato e polarizzato. Da un lato, è dominato da poche grandi piattaforme generaliste (es. Amazon, eBay) dove i volumi vengono venduti e catalogati insieme a migliaia di altre categorie di prodotti, trattando il libro come un bene di consumo qualsiasi. Dall'altro lato, le librerie indipendenti, i piccoli editori e le librerie di quartiere spesso si limitano alla sola vendita fisica o a soluzioni digitali improvvisate (es. ordini via email o social media), non disponendo né del know-how tecnico né del budget per sviluppare e mantenere un'architettura software di e-commerce proprietaria, sicura e performante.

Questa situazione genera forti inefficienze per i venditori (librai e gestori). Sulle piattaforme generaliste, i librai devono affrontare commissioni elevate e si ritrovano a competere in un ambiente dominato da algoritmi che favoriscono i grandi distributori. Inoltre, l'assenza di un pannello amministrativo pensato appositamente per le dinamiche dell'editoria rende complessa e dispendiosa in termini di tempo la gestione del magazzino (es. tracking delle disponibilità, gestione dei preordini o delle edizioni specifiche di uno stesso titolo). Senza un sistema centralizzato, la gestione del catalogo e l'elaborazione degli ordini sono lente e soggette a errori umani.

Dal punto di vista dell'utente finale (il lettore appassionato), l'esperienza risulta dispersiva e impersonale. Sui grandi marketplace, la ricerca di un libro è costantemente interrotta da suggerimenti di prodotti non pertinenti (elettronica, abbigliamento, ecc.), mentre la ricerca di titoli di nicchia, edizioni rare o volumi di piccoli editori risulta spesso frustrante. Manca, in sostanza, un ecosistema digitale unificato e ottimizzato che funga da "libreria virtuale" dedicata esclusivamente a chi cerca, esplora e acquista libri in modo mirato e senza distrazioni.

2. Obiettivi

L'obiettivo principale del progetto è fornire **WeBooks**, un'infrastruttura software unificata e dedicata esclusivamente al mercato editoriale. La piattaforma e-commerce mira a colmare il divario tra l'esperienza dispersiva dei grandi marketplace e i limiti tecnici delle librerie indipendenti, creando un ecosistema digitale su misura per chi vende e chi legge.

Nello specifico, il sistema dovrà raggiungere i seguenti sotto-obiettivi:

- **Ottimizzazione lato Venditore (Gestione):** Fornire a librai e amministratori un pannello gestionale (lato server) intuitivo e centralizzato. L'obiettivo è automatizzare e semplificare le operazioni di routine: dall'inserimento rapido di nuovi volumi a catalogo (con dettagli specifici come ISBN, edizione e casa editrice), fino all'aggiornamento in tempo reale delle giacenze di magazzino e al tracciamento efficiente dello stato degli ordini, riducendo così i costi operativi e il margine di errore.
- **Focalizzazione lato Lettore (Esplorazione):** Fornire agli utenti finali un'interfaccia client dedicata, priva di distrazioni commerciali non pertinenti. L'obiettivo è garantire un'esperienza d'uso (User Experience) fluida che permetta di cercare, filtrare e scoprire

libri in modo mirato, accedere a schede prodotto dettagliate, gestire agevolmente il proprio carrello e completare il processo di acquisto (checkout) in modo rapido e sicuro.

- **Indipendenza e Scalabilità:** Creare un'architettura software strutturata a tre livelli (three-tier) che svincoli la logica di presentazione da quella applicativa e di gestione dei dati. Questo obiettivo garantisce che il sistema sia manutenibile, scalabile nel tempo e adattabile a futuri incrementi di traffico o nuove funzionalità, fornendo alle librerie un'alternativa solida e proprietaria rispetto alle piattaforme di terze parti.

3. Scenari

Gli scenari descrivono esempi concreti e sequenziali di utilizzo del sistema dal punto di vista degli attori. Questi esempi pratici ("Vertical Slice") aiutano a definire i confini esatti di WeBooks e a chiarire le interazioni previste tra l'utente e l'applicazione, coprendo sia i casi d'uso standard (as-is/visionary) sia eventuali eccezioni.

Scenario 1: Acquisto standard di un libro (Flusso Principale)

- **Nome scenario:** `acquistoLibro_Utente`
- **Attori partecipanti:** Mario (Lettore)
- **Flusso di eventi:**
    1. Mario accede all'homepage di WeBooks tramite il suo web browser e utilizza la barra di ricerca digitando "Il Nome della Rosa".
    2. Il sistema interroga il DBMS e restituisce una pagina con i risultati pertinenti, mostrando copertina, titolo, autore e prezzo.
    3. Mario seleziona l'edizione desiderata per accedere alla scheda prodotto e leggerne la trama e la disponibilità.
    4. Soddisfatto, clicca sul pulsante "Aggiungi al carrello".
    5. Il sistema aggiorna dinamicamente l'icona del carrello mostrando un badge con il numero "1".
    6. Mario clicca sul carrello, verifica il riepilogo dell'ordine e preme "Procedi al Checkout".
    7. Mario inserisce l'indirizzo di spedizione, i dati della carta di credito e conferma l'acquisto.
    8. Il sistema valida i dati, processa il pagamento, decrementa di 1 la giacenza del libro nel database e mostra a Mario una pagina di "Ordine Confermato" con relativo ID univoco, inviando contemporaneamente una ricevuta via email.

Scenario 2: Gestione catalogo e inserimento nuova uscita

- **Nome scenario:** `inserimentoNuovoLibro_Admin`
- **Attori partecipanti:** Luigi (Amministratore/Libraio)
- **Flusso di eventi:**
    1. Luigi naviga verso l'URL dell'area amministrativa di WeBooks e inserisce le proprie credenziali (username e password).

2. Il sistema autentica Luigi e mostra la dashboard gestionale con il riepilogo delle vendite e degli ordini.
3. Luigi clicca sulla sezione "Catalogo" e poi su "Aggiungi nuovo libro".
4. Il sistema presenta un form di inserimento dati.
5. Luigi compila i campi essenziali: Titolo, Autore, ISBN, Casa Editrice, Prezzo, Categoria e imposta la "Quantità in magazzino" a 20. Carica inoltre l'immagine della copertina.
6. Luigi clicca sul pulsante "Salva e Pubblica".
7. Il sistema controlla che l'ISBN non sia duplicato e che tutti i campi obbligatori siano formattati correttamente.
8. Il sistema aggiorna il database e notifica a Luigi il successo dell'operazione. Il nuovo libro è ora immediatamente ricercabile e acquistabile dai clienti.

Scenario 3: Gestione di un'eccezione (Libro Esaurito durante il checkout)

- **Nome scenario:** `acquistoFallito_GiacenzaZero`
- **Attori partecipanti:** Mario (Lettore)
- **Flusso di eventi:**
    1. Mario ha aggiunto "Il Signore degli Anelli" al carrello (attualmente con giacenza = 1).
    2. Mentre Mario inserisce lentamente i dati di spedizione, un altro utente acquista l'ultima copia disponibile di quel libro. La giacenza nel database scende a 0.
    3. Mario clicca su "Conferma Acquisto".
    4. Il sistema tenta di processare l'ordine, ma prima di addebitare la carta verifica nuovamente la giacenza del prodotto.
    5. Il sistema rileva che la disponibilità è 0.
    6. Il sistema interrompe la transazione di pagamento (nessun addebito viene effettuato).
    7. Il sistema reindirizza Mario alla pagina del carrello, mostrando un messaggio di errore evidenziato: "Siamo spiacenti, ma il libro 'Il Signore degli Anelli' è appena andato esaurito. Rimuovi l'articolo per continuare o salvalo nella tua Lista Desideri".
    8. L'ordine non viene creato.

4. Requisiti Funzionali

I requisiti funzionali descrivono le interazioni specifiche che il sistema deve supportare per soddisfare gli obiettivi prefissati, indipendentemente dalle scelte tecnologiche di implementazione. Il sistema identifica due tipologie principali di attori, ognuno con accesso a specifiche aree di funzionalità:

4.1 Funzionalità per l'attore Lettore (Cliente)

L'attore Lettore interagisce con l'interfaccia pubblica della piattaforma. Il sistema deve permettere a questo utente di:

- **[RF-01] Ricerca e Navigazione:** Effettuare ricerche mirate all'interno del catalogo (es. per titolo, autore, genere o ISBN) e visualizzare i risultati.
- **[RF-02] Consultazione Dettagli:** Accedere a schede prodotto dettagliate per ogni volume, che mostrino informazioni essenziali come trama, prezzo, disponibilità in magazzino e copertina.
- **[RF-03] Gestione Carrello:** Aggiungere o rimuovere volumi da un carrello virtuale, visualizzando il totale parziale e il riepilogo degli articoli selezionati.
- **[RF-04] Checkout e Ordini:** Procedere all'acquisto confermando i prodotti nel carrello, inserendo i dati necessari per la spedizione e il pagamento, e ricevendo un feedback di conferma (numero d'ordine) a transazione completata.

4.2 Funzionalità per l'attore Amministratore (Libraio/Gestore)

L'attore Amministratore interagisce con l'area riservata (gestionale) del sistema. Il sistema deve permettere a questo utente di:

- **[RF-05] Gestione Catalogo (Inserimento):** Inserire nuovi libri nel database compilando form dedicati con i dettagli del volume (titolo, autore, prezzo, codice ISBN, ecc.).
- **[RF-06] Aggiornamento Magazzino:** Modificare i dati dei libri già presenti (ad esempio, per aggiornare il prezzo o modificare il numero di copie disponibili in stock).
- **[RF-07] Monitoraggio Ordini:** Visualizzare l'elenco degli ordini effettuati dai lettori e poterne aggiornare lo stato (es. "In lavorazione", "Spedito").

5. Requisiti Non Funzionali

In conformità al modello FURPS+, i requisiti non funzionali (che descrivono aspetti del sistema non legati al comportamento funzionale diretto) e i vincoli di progetto (pseudo-requisiti) sono definiti come segue:

- **[RNF-01] Performance (Prestazioni):** Il sistema deve garantire tempi di risposta rapidi durante la navigazione del catalogo, supportando la concorrenza di più utenti contemporaneamente. I tempi di caricamento delle schede prodotto non devono ostacolare o rallentare l'esperienza di acquisto.
- **[RNF-02] Usability (Usabilità):** L'interfaccia client (lato Lettore) deve essere intuitiva, accessibile e non richiedere alcuna conoscenza pregressa o manuale utente per essere navigata. La procedura di ricerca e il processo di checkout devono essere snelli e chiari.
- **[RNF-03] Reliability (Affidabilità):** Il sistema deve garantire la coerenza dei dati (es. la quantità di libri nel carrello non deve mai superare la disponibilità reale in magazzino) ed evitare blocchi applicativi durante la fase cruciale del pagamento.
- **[RNF-04] Implementation e Architecture (Vincoli di Implementazione):** Il sistema software dovrà basarsi rigidamente su un'architettura **three-tier** (tre livelli). I livelli saranno strutturati in: un client (livello di presentazione), un server (logica applicativa) e un DBMS (gestione dei dati persistenti).

6. Ambiente Target

Il sistema *WeBooks* è concepito come una Web Application distribuita, progettata per operare in un ambiente client-server accessibile tramite rete Internet. L'ambiente operativo di destinazione prevede la seguente configurazione tecnologica e di deployment:

- Ambiente Client (Utenti e Amministratori): Il sistema sarà fruibile tramite qualsiasi Web Browser moderno conforme agli standard HTML5, CSS3 e JavaScript (es. Google Chrome, Mozilla Firefox, Apple Safari, Microsoft Edge), sia da dispositivi desktop che mobili, senza necessità di installare software aggiuntivo lato utente.

- Ambiente Server (Logica Applicativa): Il livello intermedio sarà eseguito su un Web/Application Server compatibile con l'ecosistema Java Enterprise (es. Apache Tomcat), incaricato di elaborare le richieste HTTP/HTTPS, gestire le sessioni utente e orchestrare la logica di business.

- Ambiente di Persistenza (Database): La memorizzazione persistente dei dati relativi a catalogo, utenti e ordini sarà affidata a un sistema di gestione di basi di dati relazionali (RDBMS, es. MySQL), interrogato dal livello applicativo.

- Configurazione di Deployment: I tre livelli dell'architettura (Web Client, Application Server, Database Server) saranno progettati in maniera modulare e disaccoppiata. Tale scelta permette di installare ed eseguire ciascun livello su nodi hardware distinti e indipendenti, facilitando la manutenzione e garantendo la scalabilità del sistema all'aumentare del carico di lavoro.

7. Consegne e Scadenze

	Fase	**Scadenza**
1	Formazione gruppo del progetto	01/10/2026
2	Start-up progetto	08/10/2026
3	Problem Statement	15/10/2026
4	Requisiti e casi d’uso	29/10/2026
5	Requirements Analysis Document	12/11/2026
6	System Design Document	26/11/2026
7	Specifica delle Interfacce dei moduli del sottosistema da implementare	17/12/2026
8	Piano di test di sistema e specifica dei casi di test	17/12/2026
9	Consegna finale del progetto	15/01/2027